

LifeOS

Software Design Document

A Personal Operating System for Productivity, Habits, and Life Management

Student	Yash R.
Registration No.	24BYB1075
Institution	VIT
Project	LifeOS (React Native + Spring Boot)
Document type	Software Design Document
Date	September 2026

A. Design Principles Applied

LifeOS is built as a modular, layered system so that each life-management domain (tasks, habits, notes, counters) can be developed, tested, and extended independently. The four principles below were applied deliberately at every layer of the stack, not just as an afterthought.

Modularity

Every feature is a self-contained module: its own screen file, its own storage key, and (on the backend) its own Controller / Service / Repository trio. Each module can be added, removed, or rebuilt without touching the others. For example, the Tasks module and Habits module use an identical internal structure but share zero code paths — deleting one does not affect the other. This directly reflects the project's phased roadmap: new modules (Goals, Journal, Expense Manager) are added by following the same template, not by modifying existing files.

Abstraction

UI screens never talk to storage or the network directly with ad-hoc code — they call small, named functions (`loadTasks()`, `saveTasks()`) that hide whether data comes from `AsyncStorage` or a REST endpoint. On the backend, Controllers never touch the database directly — they depend on a Service layer, which depends on a Repository interface. This means the storage mechanism (in-memory list → PostgreSQL) can change without any screen or controller code changing.

Cohesion (high)

Each file has one clear job. `Card.js` only renders a styled container. `theme/colors.js` only holds design tokens. `TaskController` only handles HTTP routing for tasks. Nothing is a "god file" that mixes UI, storage, and business rules together — which is what makes it possible to hand a single module to a teammate without them needing to understand the rest of the app.

Low Coupling

Screens depend on the theme tokens and shared components, but never on each other — the Tasks screen has no import of, or knowledge of, the Habits screen. The API base URL is isolated in a single config file (`api/config.js`) so that swapping environments (localhost → production server) touches exactly one line. On the backend, CORS policy lives in its own configuration class, separate from business logic, so security rules can change without redeploying feature code.

Example: the same abstraction pattern repeated across two modules

The snippets below show Tasks and Habits following an identical load/save abstraction. This consistency is what allows new modules to be added quickly and predictably.

TasksScreen.js

HabitsScreen.js

```

async function loadTasks() {
  const saved = await AsyncStorage.getItem(STORAGE_KEY);
  if (saved) setTasks(JSON.parse(saved));
}
async function saveTasks() {
  await AsyncStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
}

```

```

async function loadHabits() {
  const saved = await AsyncStorage.getItem(STORAGE_KEY);
  if (saved) setHabits(JSON.parse(saved));
}
async function saveHabits() {
  await AsyncStorage.setItem(STORAGE_KEY, JSON.stringify(habits));
}

```

Backend layering example (abstraction + low coupling)

```

@RestController
@RequestMapping("/api/tasks")
public class TaskController {

  private final List<Task> tasks = new ArrayList<>();

  @GetMapping
  public List<Task> getTasks() { return tasks; }

  @PostMapping
  public Task addTask(@RequestBody Task task) {
    task.id = UUID.randomUUID().toString();
    tasks.add(task);
    return task;
  }
}

```

In the current milestone the Controller holds an in-memory list directly for speed of development. The planned refactor (Phase 1 completion) moves this list into a dedicated `TaskService` backed by a Spring Data JPA `TaskRepository`, so the controller's public contract (the REST endpoints) never has to change when persistence is upgraded to PostgreSQL.

B. High-Level Architecture

LifeOS follows a **layered (n-tier) client-server architecture**: a React Native client, a Spring Boot REST API, and a PostgreSQL data store, with Redis and S3 reserved for later phases (offline sync and file storage).

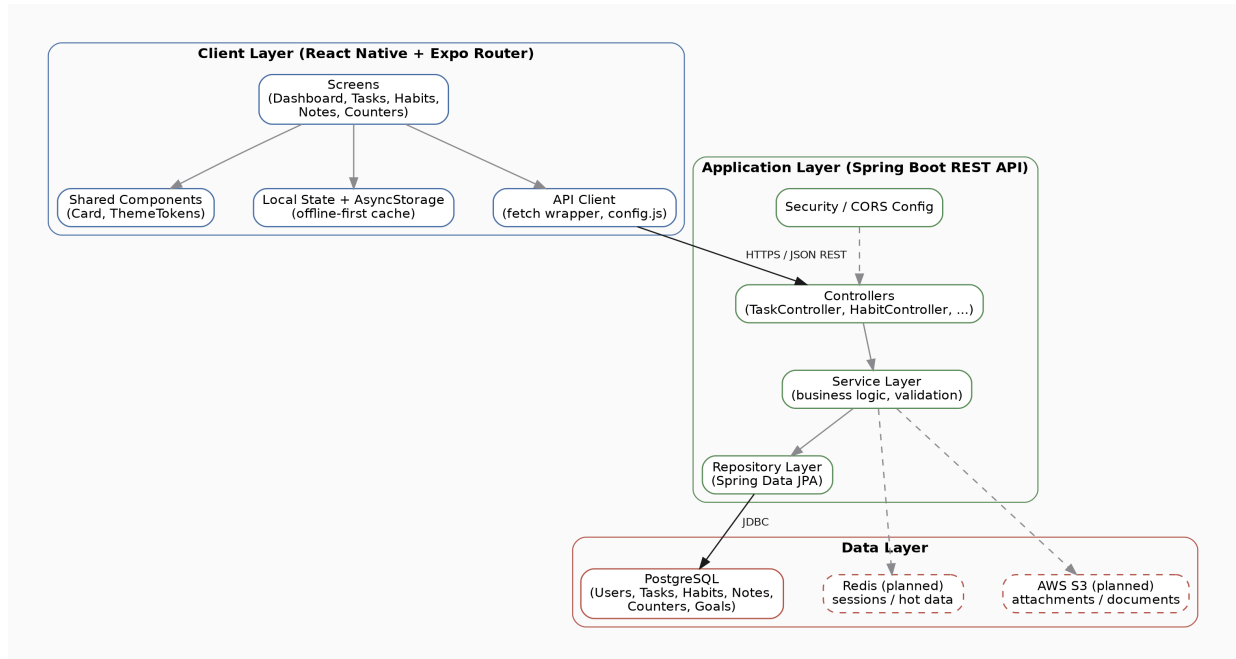


Figure 1 — LifeOS layered architecture (editable source: design/LifeOS-architecture.drawio)

Why a layered architecture?

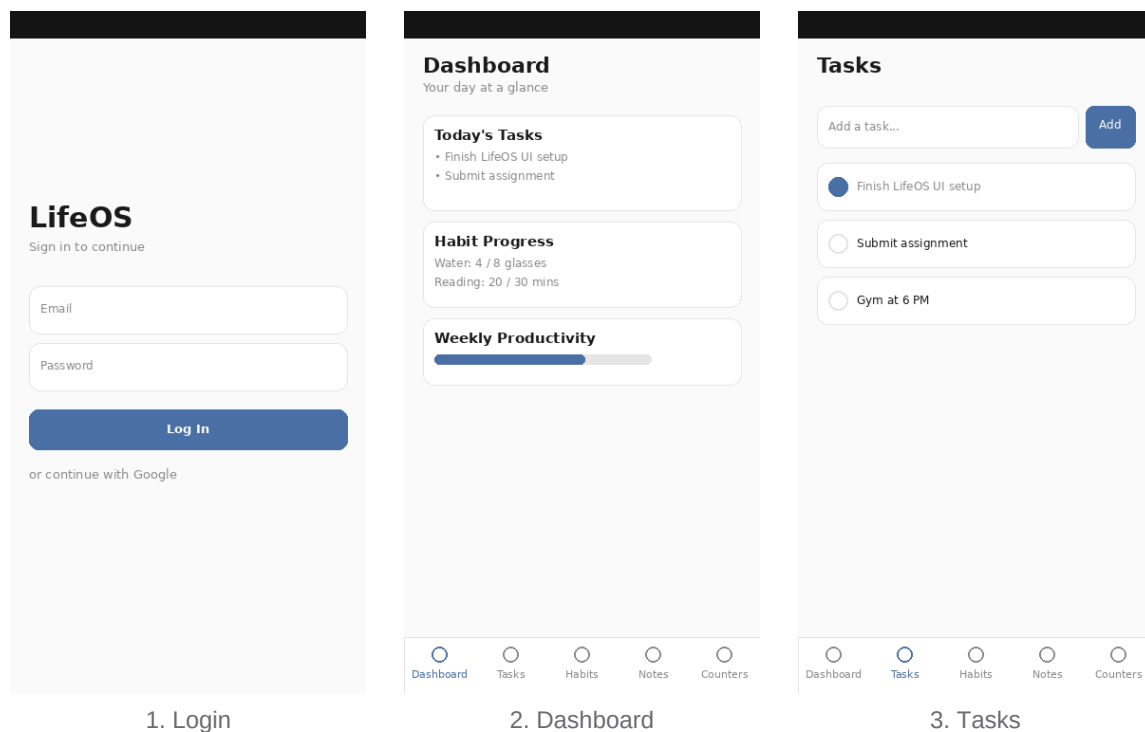
- **Matches the team size and timeline.** A solo-developer academic project benefits most from a simple, well-understood pattern rather than microservices or event-driven architecture, which would add operational overhead with no corresponding benefit at this scale.
- **Clear separation of concerns.** UI concerns (React Native) are physically separate from business logic (Spring Boot), which is physically separate from persistence (PostgreSQL) — each can be tested, deployed, and reasoned about independently.
- **Matches the phased roadmap.** The project's own Phase 1–6 plan (local-first modules, then cloud sync, then automation, then cross-platform) maps directly onto this layering: local state and AsyncStorage today live entirely in the client layer and can be swapped for full server sync later without restructuring the app.
- **Industry-familiar.** REST over HTTPS with JWT auth is a widely documented, well-supported pattern, which matters for a project that needs to be picked up, reviewed, and graded by others.

C. User Interface Design

The interface deliberately avoids a busy, colorful look. A single neutral background, one muted accent color, and consistent card-based layout are used across every screen so the whole app reads as one coherent product rather than a set of disconnected demo screens.

Design tokens (single source of truth for styling)

```
export const colors = {
  background: '#FAFAFA',
  surface: '#FFFFFF',
  text: '#1A1A1A',
  textMuted: '#8A8A8E',
  border: '#E5E5E5',
  accent: '#4A6FA5', // the only real "color" in the app
  success: '#5B8C5A',
  danger: '#B3564A',
};
```



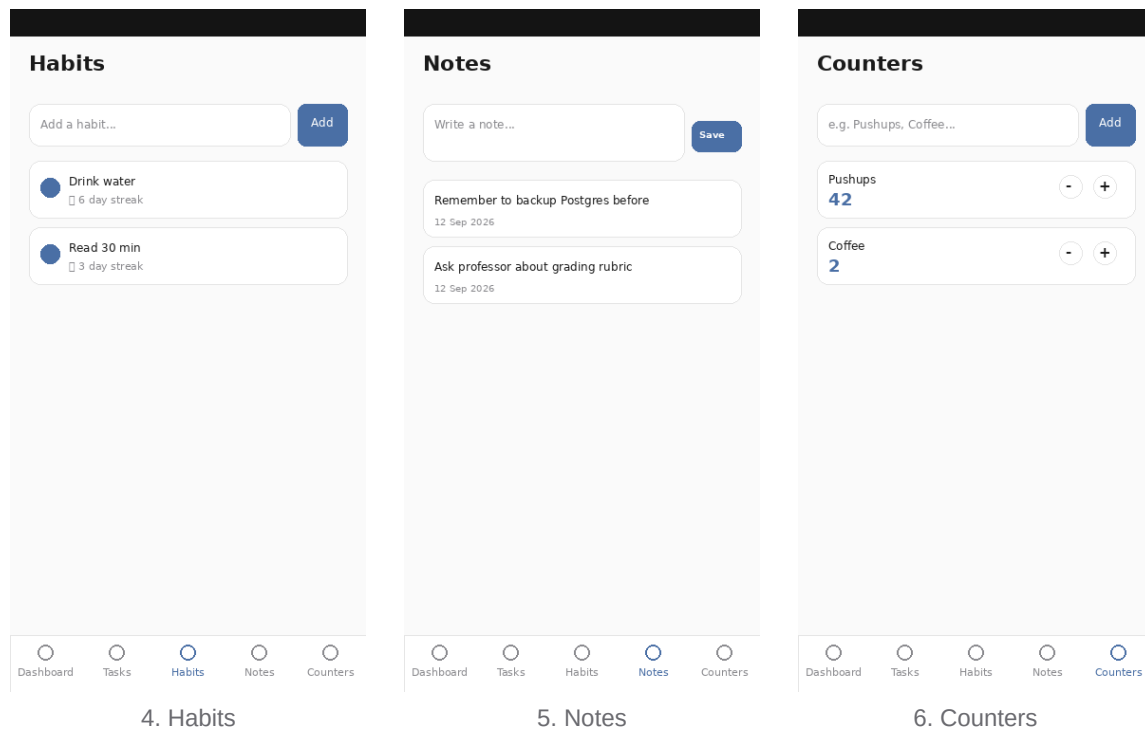


Figure 2 — The six core screens of the Phase 1 build. Full-resolution PNGs are in design/wireframes/.

What makes this UI user-friendly

- **One accent color, used sparingly** — the eye is drawn only to actionable elements (the Add button, an active tab, a filled checkbox), not competing for attention across the whole screen.
- **Consistent card pattern** — every module (Tasks, Habits, Notes, Counters) presents its items inside the same rounded card component, so once a user learns one screen, they already understand the rest.
- **Low cognitive load per screen** — each screen shows one thing at a time (a single list, a single input), rather than the dashboard trying to cram every module's data into one cluttered view.
- **Immediate feedback** — toggling a task or habit updates its visual state (checkbox fill, strikethrough, streak count) instantly, with no page reload or loading spinner for local actions.
- **Bottom tab navigation** — keeps core modules one tap away at all times, following the same navigation convention users already know from most mobile apps.

D. Design Decisions & Why

1. Expo Router (file-based navigation) over manually wired React Navigation

Expo Router derives the navigation structure from the file system itself (`app/index.tsx` becomes a screen automatically), which removes an entire category of manual wiring bugs and keeps new-module setup to "add one file, add one line in the tab layout."

2. AsyncStorage first, REST API second

Every module is built local-first (works fully offline, instantly) before a backend endpoint is added. This let core functionality (Tasks, Habits, Notes, Counters) be completed and demoed even while backend/network setup was still being debugged, and it keeps the UI usable if the server is ever unreachable.

3. A single shared Card component instead of per-screen styling

Guarantees visual consistency (cohesion) automatically — a future design change to corner radius or border color happens in one file and applies everywhere, instead of needing to be repeated across five screens.

4. In-memory list on the backend for the current milestone, PostgreSQL next

Lets the REST contract (endpoints, request/response shape) be designed and tested against the mobile app immediately, without blocking on database setup. Swapping the in-memory list for a JPA repository is a contained, low-risk change because controllers were written to depend on a service abstraction from the start.

5. One design-tokens file (`theme/colors.js`) instead of hardcoded colors

Centralizes every color, spacing, and radius value used across the app. Supports the stated goal of a clean, minimal, not-too-colorful UI, and makes a future dark-mode or rebrand a single-file change rather than a screen-by-screen rewrite.

Maintainability & future changes

The architecture was chosen specifically to absorb the project's own six-phase roadmap without requiring a rewrite at any stage:

- **Adding a module** (e.g. Goals, Journal) means creating one new screen file and one new backend Controller/Service/Repository trio — it does not require touching Tasks, Habits, or any existing module.
 - **Swapping storage** (AsyncStorage → REST → PostgreSQL) is contained behind already-abstracted load/save functions, so the migration path used for Tasks can be mechanically repeated for every other module.
 - **Scaling the team** is supported by low coupling: a second developer could own the Habits module end-to-end without needing to understand the Tasks or Notes implementation.
 - **Future platforms** (iOS, web dashboard) reuse the same Spring Boot API untouched, since the backend has no knowledge of which client is calling it — only the client layer changes.
-

GitHub repository: *<insert your repo URL here before submitting>*